

UNIVERSIDAD
DE GRANADA

Artificial Bee Colony

Práctica opcional

Pablo Linari Pérez

DNI: 75571079W
Curso: 25/26
Correo: pablolinari@correo.ugr.es

Índice de contenidos

1 Funcionamiento del algoritmo 3
1.1 La metáfora de la colonia 3
1.2 Parámetros del algoritmo 3
1.3 Ecuaciones fundamentales 4
1.4 Etapas del algoritmo 4
1.4.1 Etapa 1: Inicialización 4
1.4.2 Etapa 2: Fase de abejas empleadas 4
1.4.3 Etapa 3: Fase de abejas observadoras 4
1.4.4 Etapa 4: Fase de abeja exploradora 5
1.5 Pseudocódigo 5
1.6 Adaptaciones al problema del portfolio 6

1 Funcionamiento del algoritmo

El algoritmo **Artificial Bee Colony** (ABC), propuesto por Karaboga y Basturk en 2007, es una metaheurística poblacional inspirada en el comportamiento de búsqueda de alimento de las colonias de abejas. La idea central es que un conjunto de abejas explora el espacio de búsqueda en torno a una serie de **fuentes de alimento**, intensificando la exploración alrededor de las fuentes más prometedoras y abandonando aquellas que se agotan.

En el contexto de este problema cada **fuente de alimento** representa una solución candidata (un vector de pesos del portfolio), y la **cantidad de néctar** de una fuente se corresponde con el valor de fitness de esa solución. El objetivo del algoritmo es, por tanto, encontrar la fuente con mayor cantidad de néctar, es decir, la solución que maximiza la función objetivo.

1.1 La metáfora de la colonia

La colonia se divide en tres tipos de abejas, cada uno con un papel distinto dentro de la búsqueda:

- **Abejas empleadas (employed bees)**: cada una está asociada a una fuente de alimento concreta y se encarga de explotarla, buscando soluciones mejores en su vecindario.
- **Abejas observadoras (onlooker bees)**: esperan en la colmena y eligen qué fuentes explotar en función de la información que comparten las empleadas. Las fuentes con más néctar tienen más probabilidad de ser elegidas, lo que concentra el esfuerzo en las zonas prometedoras (intensificación).
- **Abejas exploradoras (scout bees)**: cuando una fuente se agota (no mejora durante muchos ciclos) se abandona, y la abeja empleada asociada se convierte en exploradora, buscando una fuente completamente nueva de forma aleatoria (diversificación).

El número de fuentes de alimento SN es la mitad del tamaño total de la colonia, y coincide tanto con el número de abejas empleadas como con el de abejas observadoras (cada fuente tiene una empleada asociada).

1.2 Parámetros del algoritmo

El comportamiento del algoritmo se controla mediante los siguientes parámetros:

- **colonySize**: tamaño total de la colonia. El número de fuentes de alimento se calcula como $SN = \frac{\text{colonySize}}{2}$. En la implementación se usa un valor de 50, por lo que $SN = 25$.
- **trialLimit** (parámetro *limit* del artículo): número máximo de ciclos que una fuente puede permanecer sin mejorar antes de ser abandonada. Controla cuándo entra en juego la fase exploradora.
- **maxevals**: número máximo de evaluaciones de la función objetivo, que actúa como criterio de parada.

1.3 Ecuaciones fundamentales

El algoritmo se apoya en dos ecuaciones clave. La primera genera una **solución candidata** v_i a partir de una fuente existente x_i, perturbando una única componente j elegida al azar:

$$v_{ij} = x_{ij} + \phi_{ij} \cdot (x_{ij} - x_{kj})$$

donde $k \neq i$ es otra fuente vecina elegida aleatoriamente y ϕ_{ij} es un número aleatorio en el intervalo $[-1, 1]$. El resto de componentes de v_i se copian directamente de x_i. Intuitivamente, la nueva solución se acerca o se aleja de una fuente vecina en una sola dimensión.

La segunda ecuación calcula la **probabilidad** de que cada fuente sea seleccionada por una observadora, de forma proporcional a su néctar:

$$p_i = \frac{\text{fit}_i}{\sum_{n=1}^{SN} \text{fit}_n}$$

Cuanto mayor es el fitness de una fuente, mayor es su probabilidad de ser explotada por las observadoras.

1.4 Etapas del algoritmo

La ejecución del algoritmo se organiza en una etapa inicial de inicialización seguida de un bucle de ciclos. Cada ciclo se compone de tres fases consecutivas (empleadas, observadoras y exploradoras), que se repiten hasta agotar el número máximo de evaluaciones.

1.4.1 Etapa 1: Inicialización

Se generan SN fuentes de alimento de forma aleatoria mediante `createSolution()`, asegurando que cada una cumple las restricciones del problema (pesos en $[\text{low}, \text{high}]$ que suman 1). Se evalúa el fitness de cada una y se memoriza la mejor solución encontrada hasta el momento. Además, se inicializa a 0 un contador de *trials* por cada fuente, que registra cuántos ciclos lleva sin mejorar.

1.4.2 Etapa 2: Fase de abejas empleadas

Cada abeja empleada explota su propia fuente x_i. Para ello genera una solución candidata v_i aplicando la ecuación (2.2) y la evalúa. A continuación se aplica una **selección greedy**: si la candidata es mejor que la fuente original, la sustituye y el contador de *trials* de esa fuente se reinicia a 0; en caso contrario, la fuente se mantiene y su contador de *trials* se incrementa. Esta fase explora el vecindario de todas las fuentes existentes.

1.4.3 Etapa 3: Fase de abejas observadoras

Primero se calculan las probabilidades de selección de cada fuente con la ecuación (2.1). Después, se reparten SN observadoras: cada una elige una fuente mediante **selección por ruleta** (las fuentes con más néctar tienen más probabilidad) y la explota igual que una empleada, generando una candidata con la ecuación (2.2) y aplicando de nuevo la

selección greedy. Como las mejores fuentes tienden a ser elegidas varias veces, esta fase intensifica la búsqueda en las regiones más prometedoras del espacio.

1.4.4 Etapa 4: Fase de abeja exploradora

Al final de cada ciclo se comprueba si alguna fuente se ha agotado. Se localiza la fuente con mayor número de *trials* acumulados y, si este supera el `trialLimit`, dicha fuente se abandona: se sustituye por una nueva solución aleatoria generada con `createSolution()`, se evalúa y su contador de *trials* vuelve a 0. Se permite **como máximo una** abeja exploradora por ciclo. Esta fase introduce diversificación y evita que el algoritmo quede estancado en óptimos locales.

En las tres fases se actualiza siempre la mejor solución global encontrada, que es la que finalmente devuelve el algoritmo.

1.5 Pseudocódigo

```
funcion ABC(problema, maxevals):
  SN = colonySize / 2
  # --- Inicialización ---
  para i en 0..SN:
    fuentes[i]   = problema.createSolution()
    fitness[i]   = problema.fitness(fuentes[i])
    trials[i]    = 0
    actualizar_mejor(fuentes[i], fitness[i])

  # --- Ciclos de búsqueda ---
  mientras evals < maxevals:

    # (a) Fase empleadas
    para i en 0..SN:
      candidata = produceCandidate(fuentes, i)   # ec. 2.2
      si fitness(candidata) > fitness[i]:         # selección greedy
        fuentes[i] = candidata;  trials[i] = 0
      sino:
        trials[i] += 1
      actualizar_mejor(candidata)

    # (b) Fase observadoras
    probs = computeProbabilities(fitness)         # ec. 2.1
    para t en 0..SN:
      i = rouletteSelect(probs)
      candidata = produceCandidate(fuentes, i)    # ec. 2.2
      si fitness(candidata) > fitness[i]:         # selección greedy
        fuentes[i] = candidata;  trials[i] = 0
      sino:
        trials[i] += 1
      actualizar_mejor(candidata)

    # (c) Fase exploradora (máx. una por ciclo)
    peor = indice con mayor trials
    si trials[peor] >= trialLimit:
      fuentes[peor] = problema.createSolution()
```

```
        trials[peor]  = 0
        actualizar_mejor(fuentes[peor])

    devolver mejor_solucion
```

1.6 Adaptaciones al problema del portfolio

La formulación original de ABC está pensada para la optimización de funciones numéricas sin restricciones, por lo que ha sido necesario realizar dos adaptaciones:

- **Reparación de soluciones**: la ecuación (2.2) perturba una componente y, en general, rompe las restricciones del problema (los pesos deben sumar 1 y estar en $[\text{low}, \text{high}]$ o ser 0). Por ello, tras generar cada candidata se comprueba su validez y, si no es factible, se repara con `problem.fix()` antes de evaluarla.
- **Desplazamiento de las probabilidades**: la ecuación (2.1) asume cantidades de néctar no negativas, pero el fitness del portfolio (beneficio menos λ por riesgo) puede ser negativo. Para poder aplicar la ruleta, se desplazan todos los valores de fitness sumándoles una constante que deja el peor de ellos ligeramente por encima de 0, conservando las diferencias relativas entre fuentes. Si todas las fuentes tienen el mismo fitness se reparte la probabilidad de forma uniforme.